Python Engineering - Class 03

Loops and Iteration | EX05-EX08 - Loops and Explicit Euler

Class objective: Learn when a problem requires repetition, how to control an iteration, and how iteration becomes a numerical algorithm.

This document is the base exercise guide. It is not a full solution to copy: it organizes the problem, implementation steps, reference values and required validation.

Path

```
01_python_bootcamp/
`-- week_01_logic/class_03_loops/
    |-- notes/
    `-- exercises/
```

Required workflow

```
PROBLEM
INPUTS
OUTPUTS
EQUATIONS
SPECIAL CASES
DO I NEED ITERATION?
ALGORITHM
VALIDATION
```

Before coding, write these eight sections. The goal is to separate problem reasoning from syntax.

EX05 - Sum of Squares

Problem: Compute S=sum(i^2) from i=1 to N using an accumulator.

Reference inputs: N positive integer

Equations / relationship: S = sum_{i=1..N} i^2; analytical S=N(N+1)(2N+1)/6

Expected check: For N=3 -> 14; compare loop with analytical formula.

Implementation guide

- 1. Initialize total=0.
- 2. Loop over range(1,N+1).
- 3. Accumulate i**2.
- 4. Compare with closed-form formula.

Before running

Predict sign, order of magnitude, data type and any relevant shape. Then compare the prediction with the actual output.

EX06 - Factorial

Problem: Compute n! with ascending and descending loops.

Reference inputs: n non-negative integer

Equations / relationship: n! = n*(n-1)*...*1; 0! = 1

Expected check: n=5 -> 120; compare with math.factorial().

Implementation guide

- 1. Initialize product=1.
- 2. Iterate over factors.

- 3. Multiply accumulator.
- 4. Validate with math.factorial().

Before running

Predict sign, order of magnitude, data type and any relevant shape. Then compare the prediction with the actual output.

EX07 - Taylor Approximation of exp(x)

Problem: Approximate e^x with a finite sum of terms x^n/n!.

Reference inputs: N=5; xp=0.5

Equations / relationship: exp(x) approx sum_{n=0..N} x^n/n!

Expected check: Compare numerical approximation with math.exp(0.5). Increasing N should generally reduce truncation error near the expansion point.

Implementation guide

- 1. Include n=0 term.
- 2. Maintain factorial incrementally.
- 3. Accumulate terms.
- 4. Evaluate at xp.
- 5. Compare with exp().

Before running

Predict sign, order of magnitude, data type and any relevant shape. Then compare the prediction with the actual output.

EX08 - Vertical Motion with Euler

Problem: Integrate dh/dt=v and dv/dt=-g with explicit Euler until the ground is reached.

Reference inputs: h0=0; v0=50; t0=0; g=9.81; dt=0.1

Equations / relationship: h_{k+1}=h_k+v_k*dt; v_{k+1}=v_k-g*dt; t_{k+1}=t_k+dt

Expected check: Analytical h_max about 127.42 m; flight time about 10.19 s; final speed magnitude about 50 m/s. Numerical values depend on dt and stopping rule.

Implementation guide

- 1. Validate dt,g and initial conditions.
- 2. Initialize state.
- 3. Iterate while h>=0.
- 4. Update h,v,t.
- 5. Track maximum.
- 6. Compare with analytical solution.

Before running

Predict sign, order of magnitude, data type and any relevant shape. Then compare the prediction with the actual output.

Final validation

Answer these questions as comments at the end of the exercise. The goal is not memorized wording but demonstrating that you can explain the code flow.

- Difference between for and while.
- Why does a sum accumulator start at 0 and a product at 1?
- What does it mean that range() excludes stop?
- What does N represent in the Taylor series?
- What is dt in Euler and what generally happens when it is reduced?
- Why does a simulation need a stopping condition?

Completion criterion

The class is complete when the code runs, the reference checks make sense, you can explain every block without reading a solution, and you can modify at least one input while predicting the expected effect.